

# Person 6 — Frontend / Web-GIS / Dashboards / Human Verification

Intelligent Land Record Digitization, Validation & Urban Cadastral Mapping Platform |  
Step-by-step implementation specification | Unified Master Blueprint v2.0

---

**Assigned role:** Person 6 — Frontend / Web-GIS / Dashboards / Human Verification

**Document set:** Unified Master Blueprint v2.0

**Scope:** SIH 12 + SIH 18 combined platform

**Canonical rule:** This document must not introduce a technology, endpoint, role, state, table, or handoff that conflicts with the Master Blueprint.

# 1. Mission

Own the role-aware officer application: login/session handling, project UI, document upload/viewer, extraction/review workspace, Web-GIS, dashboards, audit views and accessible UX.

## 2. Non-negotiable shared rules

- Use the canonical stack, API paths, entities, statuses, application roles and event names from Master Blueprint v2.0.
- Do not change another person's contract by silently renaming fields or endpoints.
- All work is project-scoped through `project_id`.
- All long-running work is asynchronous with job/workflow status.
- Original documents/source imagery remain immutable.
- Every AI output carries confidence, provenance, `model_version` and timestamp.
- Every critical mutation is auditable.
- Before handoff, run tests and provide evidence.

## 3. Exact handoff principle

A handoff is complete only when the receiving person can consume the documented output without guessing field names, endpoint names, status values, storage locations, or semantics.

## 4. Required routes

| Route                 | Purpose                                             |
|-----------------------|-----------------------------------------------------|
| /login                | Authentication and session                          |
| /dashboard            | Processing, validation, progress, errors and charts |
| /projects             | Project list/creation                               |
| /documents            | Document list/upload/search/filter                  |
| /documents/:id        | Original pages, OCR, fields, confidence, provenance |
| /verification         | Review queue                                        |
| /verification/:taskId | Verification workspace                              |
| /map                  | Web-GIS                                             |
| /parcels/:id          | Parcel details                                      |
| /issues               | Validation issue queue                              |
| /audit                | Audit viewer                                        |
| /admin                | User/role administration for authorized roles       |

## 5. Step-by-step frontend implementation

- Create the React/Vite/TypeScript application and typed API client from OpenAPI.
- Implement login, token refresh and logout using the backend contract.
- Load `/api/v1/users/me` and derive allowed routes/actions from server-provided permissions.
- Implement project list and project creation with project-scoped context.
- Implement document upload with progress, validation errors and asynchronous job polling.
- Implement document viewer with page thumbnails, zoom, OCR text and evidence bounding boxes.
- Implement extraction editor showing original value, normalized value, confidence, validation errors and source page.

- Implement two-way selection: clicking a field highlights its evidence; clicking evidence selects the field.
- Implement review queue with assignment, filters, severity and status.
- Implement review workspace with original document, extracted fields, GIS evidence, validation issues and action controls.
- Disable approval when critical validation errors remain; do not rely on frontend-only enforcement.
- Implement correction flow with before/after values, reason and confirmation.
- Implement Web-GIS with imagery, draft parcels, approved parcels, buildings, roads, land use and topology-error layers.
- Implement feature search by parcel ID, survey number, village and class.
- Implement draft geometry edit with save/cancel and explicit provisional/draft status.
- Implement dashboards for processing, confidence, validation, review backlog, geographic progress and system health.
- Implement audit viewer for authorized users.
- Implement CSV/JSON/GeoJSON/GeoPackage export controls using backend endpoints.
- Implement responsive accessibility: keyboard navigation, focus states, screen-reader labels and non-color-only confidence states.

## 6. UI state model

| State                            | Required UI behavior                                        |
|----------------------------------|-------------------------------------------------------------|
| Loading                          | Show progress/placeholder; prevent duplicate action         |
| Empty                            | Explain why no data exists and provide next valid action    |
| Error                            | Show machine-readable-friendly message and retry where safe |
| Unauthorized/Forbidden           | Do not expose protected content; show permission state      |
| Processing                       | Poll job/workflow status and show current stage             |
| Low confidence                   | Show numeric confidence + accessible warning/reason         |
| Validation issue                 | Show severity, rule, message and source references          |
| Unsaved correction               | Warn before leaving                                         |
| Geometry unavailable/provisional | Clearly mark status; never imply legal authority            |
| Success                          | Confirm only after backend success response                 |

## 7. Review workflow implementation

1. Reviewer opens task.
2. Load original source and evidence.
3. Load structured fields and confidence.
4. Load validation issues.
5. Load GIS evidence where available.
6. Allow Approve/Correct/Reject/Reprocess according to server permissions.
7. For Correct, submit versioned correction and reason.
8. Refresh from backend after mutation.
9. Show audit/history result.
10. Do not enable approval if backend reports critical unresolved issues.

## 8. Web-GIS implementation

1. Initialize map using project extent and declared CRS behavior.

2. Load imagery and vector layers through authorized backend/GIS endpoints.
3. Render draft and approved layers separately.
4. Render topology errors as a distinct layer.
5. Select feature → show attributes, confidence, source asset and status.
6. Edit only draft geometry.
7. Save through PATCH endpoint; never write directly to PostGIS.
8. Refresh validation/topology state after edit.
9. Expose export only where the role has export permission.

## 9. Role-aware UI matrix

| Role     | UI focus                                                 | Do not expose as executable action                |
|----------|----------------------------------------------------------|---------------------------------------------------|
| ADMIN    | Administration, all permitted workflows                  | Nothing beyond server permission                  |
| OFFICER  | Documents, processing, operational review                | Role management, unauthorized GIS approval        |
| REVIEWER | Review/approval, validation, GIS review where authorized | User administration                               |
| SURVEYOR | Imagery, GeoAI, draft GIS editing, field evidence        | Land-record approval unless separately authorized |
| VIEWER   | Approved/read-only views and dashboards                  | Draft records, edits, corrections, approvals      |

## 10. Frontend acceptance tests

- Test every role's navigation and access restrictions.
- Upload a document and observe processing to review.
- Select field ↔ evidence highlight works.
- Correct field and verify new version after refresh.
- Approval is blocked when critical issue remains.
- Map toggles layers and edits draft geometry.
- Viewer cannot see draft layers.
- Dashboard values match API values.
- Keyboard/accessibility states work.
- API failures show usable recovery states.

## 11. Handoff

- P5 supplies OpenAPI, authentication/session and permission responses.
- P2 supplies OCR/evidence bounding-box schema.
- P3 supplies validation issue and review-task schema.
- P4 supplies GIS layer/feature/topology schemas.
- P1 supplies workflow status semantics and integration event behavior.

## 12. Definition of done

- Role-aware application works from real backend responses.
- Document review is evidence-grounded.
- GIS draft editing is versioned.

- Critical validation blocks approval.
- Dashboard metrics are backend-derived.
- Accessibility/error/loading states are implemented.
- E2E upload-to-review and map-edit tests pass.